Wine Quality Prediction Using a Weighted Ensemble

A reproduction study of Ma et al. (2024), Journal of Business Research 176:114576

SURIANANDHAN S
127018060
CSE425 — Machine Learning Essentials

Guide
Dr. Vidivelli S, AP-II
Department of CSE, School of Computing

First Review

The project in 30 seconds

Task, result, and contribution — stated up front

89.73%

REPORTED BY THE PAPER

69.01%

OUR FAITHFUL REPRODUCTION

3 checks

EXPLAIN THE GAP, FROM THE PAPER'S OWN TABLES

- Task: predict a wine's 3–8 sensory quality score from 11 physicochemical lab measurements (1,599 red wines, UCI)
- Base paper: a weighted ensemble of four learners, reporting 89.73% overall accuracy
- This work: the same method, implemented faithfully, reaches 69.01%
- The project IS that gap — it is an investigation, not an attempt to tune the number upward
- Contribution: a leak-free reproduction, a three-check diagnosis, and a terminology correction in the paper's own metric

Problem statement

Why predict a taste score from chemistry?

The expensive signal

Quality is scored 3–8 by a three-sommelier sensory panel — slow, costly, and subjective.

The cheap signal

11 physicochemical tests the winery already runs — alcohol, volatile acidity, sulphates, pH, density.

- Business relevance: that quality score drives pricing, shelf placement and inventory decisions
- The ML question: can the objective measurements predict the subjective verdict?
- Why it is hard: six classes, severely imbalanced — 1,319 of 1,599 samples sit in just two of them

Literature survey

Where this work sits

Work	Method	Reported result
Cortez et al. (2009)	SVM on the same 1,599 samples	Per-class: 0 / 20.0 / 67.5 / 57.7 / 58.6 / 0
Ma et al. (2024) — base paper	**Weighted ensemble: SVM + RNN + ELM + RF**	**89.73% overall, +9.02 over best base learner (RF 80.71%)**
This work	Same ensemble, classical-ML substitutions	69.01% + a diagnosis of the gap

- Ma et al.'s novelty: weights combine an accuracy term AND a diversity term, not accuracy alone
- Published accuracy for this exact 6-class task is typically reported in the 62–72% range
- 89.73% sits above that band — which is part of why this paper was worth reproducing

The base paper's method

What I had to re-implement

- Four base learners trained under 10-fold CV: SVM, RNN, ELM, Random Forest
- Each gets a weight $w_i = \frac{1}{2}$ (w_accuracy + w_diversity) — Eq. (4)
- Accuracy weight, Eq. (5): normalised INVERSE of each learner's CV error — fewer mistakes, more weight
- Diversity weight, Eqs. (7)–(9): a 0/1 error vector per learner; sum the Euclidean distances to every other learner's; normalise
- Combine, Eq. (1): $\hat{y} = \Sigma\, w_i\, \hat{y}_i$ — a weighted sum

Paper's own weights (their Table 8)

SVM	0.2516
RNN	0.2286
ELM	0.2168
RF	**0.3030**

Remember 0.3030 — it returns on slide 14

Objectives and success criteria

How this project should be judged

- Re-implement Ma et al.'s weighted ensemble under a classical-ML-only constraint
- Evaluate under their stated protocol: stratified 10-fold CV × 10 repeats = 100 fold-runs
- Compare per-class results against both Cortez (2009) and Ma (2024)
- Where results differ, diagnose the cause from evidence rather than tune to match

This project is successful if:

- Every component the paper specifies is implemented as written — Eqs. (1) and (4)–(9), min-max preprocessing, 10-fold CV × 10 repeats
- The evaluation is leak-free and the protocol is stated precisely enough to be re-run
- Whatever number the method produces is reported as-is, with any discrepancy investigated rather than tuned away

It is NOT successful if we reach 89.73% by changing the protocol until the number moves.

Existing system vs. proposed system

Where I followed the paper, and where I had to decide

	Existing — Ma et al. (2024)	Proposed — this work
Base learners	SVM, RNN, ELM, RF	SVM, GradientBoosting, MLP, RF
Scaling	min-max (timing unstated)	min-max inside a Pipeline, fit per training fold
Class imbalance	not stated	balanced class weights on SVM and RF
Reading of Eq. (1)	ambiguous	both soft and hard vote implemented
Verification	—	the paper's own tables reconciled in code

Where the paper is silent or ambiguous, I made the choice explicit rather than guessing quietly.

Syllabus substitutions

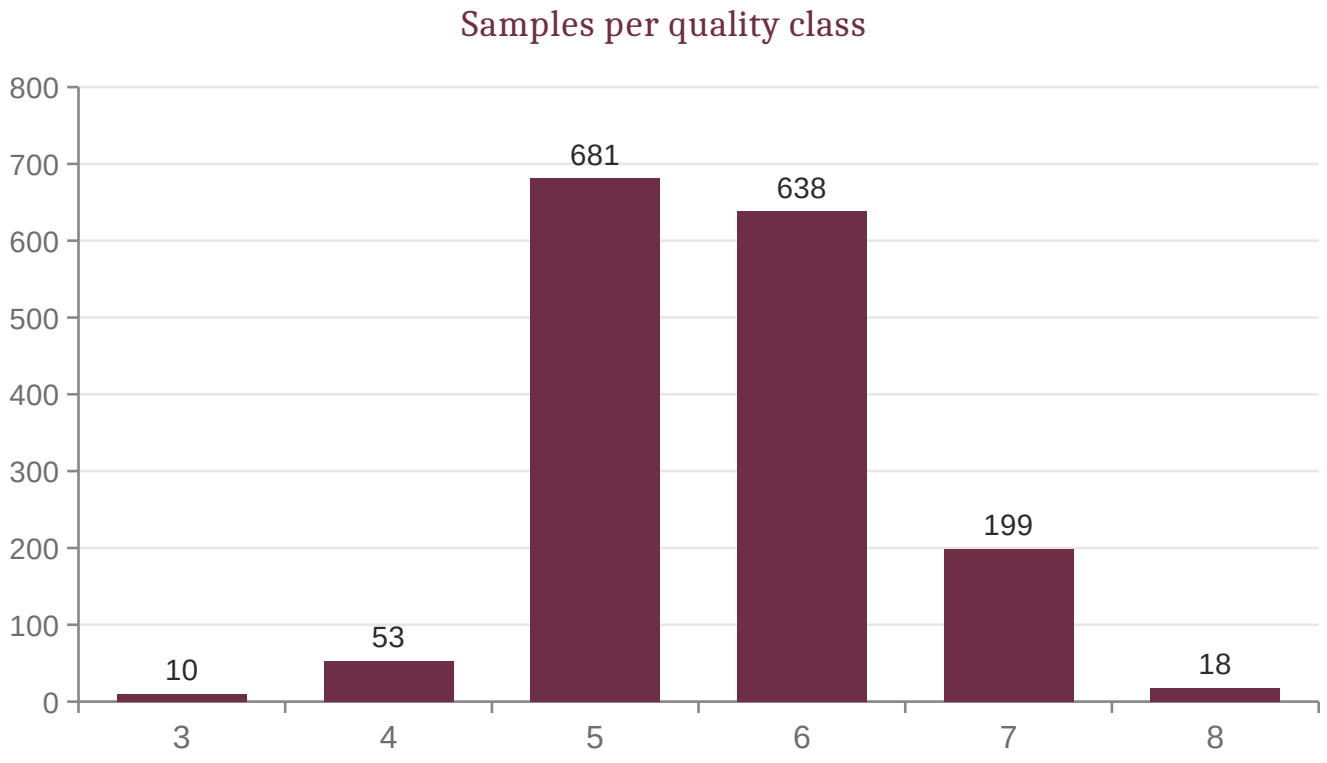
Two learners are outside a classical-ML course

Paper	Why out of scope	Substitute	Why this one
SVM	—	SVC — unchanged	already classical ML
Random Forest	—	RandomForestClassifier — unchanged	already classical ML
RNN	deep-learning sequence model; backprop-through-time	GradientBoostingClassifier	each new tree fits the residual errors of the previous ones — the in-syllabus analogue of iterative refinement
ELM	random FIXED hidden weights, closed-form output — explicitly not backprop	MLPClassifier	closest analogue that is still a neural net, but trained end-to-end by standard backpropagation

Half the ensemble — SVM and Random Forest — is unchanged, including Random Forest, the strongest member in both their run and mine.

Dataset

UCI Wine Quality (Red) — the same 1,599 samples the paper uses



- 1,599 samples · 11 features · quality 3–8
- Classes 5 and 6 alone are 82.5% of the data
- Classes 3 and 8 together are 1.75% — 10 and 18 samples
- Under 10-fold CV that is about ONE class-3 sample per test fold
- No class dropped, no class merged — all six retained, as in the paper

System architecture

One command; everything inside the box runs 100 times

winequality-red.csv
1599 × 11

preprocessing.py

Stratified 10-fold CV × 10 repeats = 100 fold-runs

SVM

GradBoost (→ RNN)

MLP (→ ELM)

RF

each wrapped in Pipeline([MinMaxScaler → estimator]) — scaler fit on the TRAIN fold only

0/1 error vector per learner (Eq. 7)

w_accuracy (Eq. 5) + w_diversity (Eqs. 8–9)

$w_i = \frac{1}{2}(w_acc + w_div)$ — Eq. (4)

Eq. (1) — two readings

soft vote: $\Sigma\, w_i \cdot P_i$

hard vote: $\Sigma\, w_i \cdot \text{onehot}(\hat{y}_i)$

pooled confusion matrix · per-class metrics · reconciliation against the paper's tables

evaluate.py

Implementation and protocol

Reproducible from one command

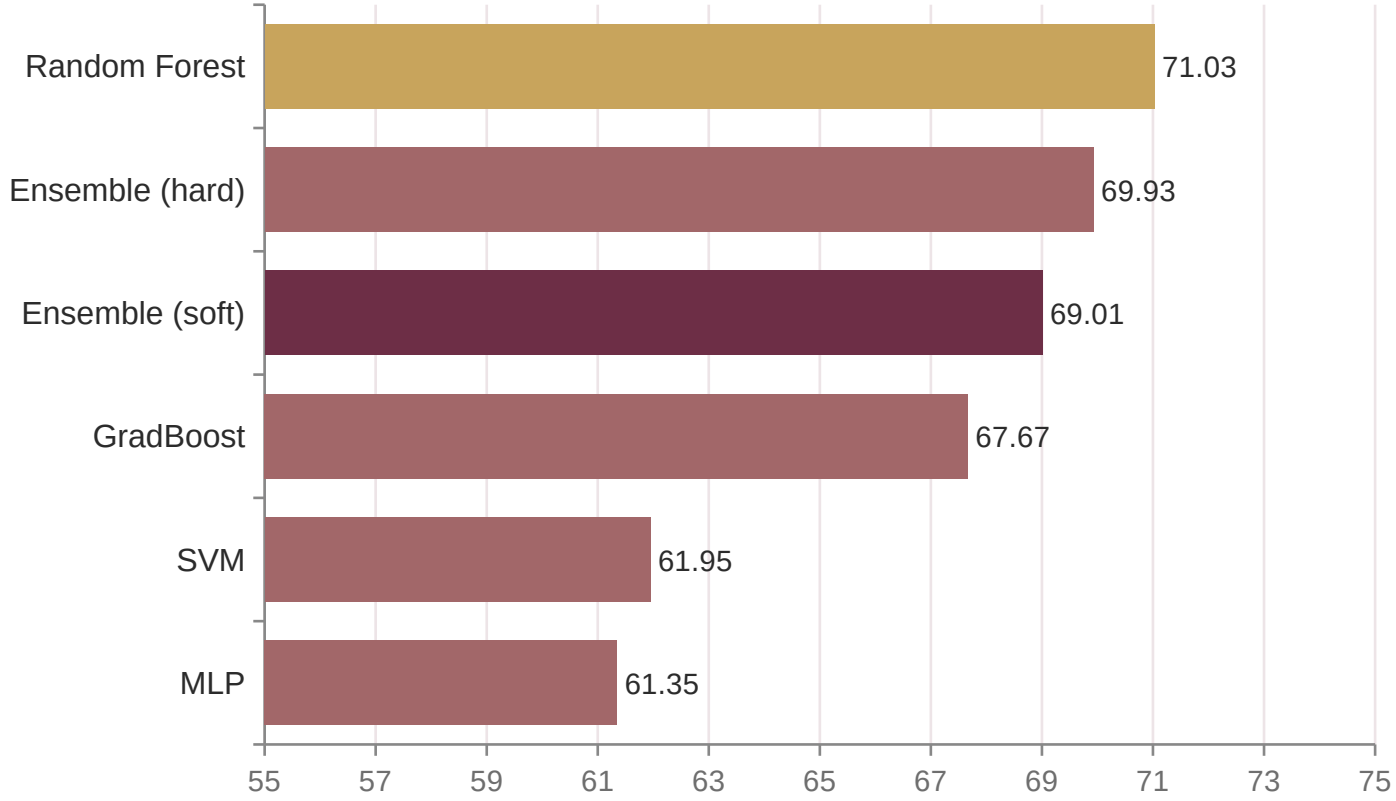
- Stack: Python 3, scikit-learn, pandas, NumPy, matplotlib. Single entry point: python run_all.py
- Protocol: stratified 10-fold CV × 10 repeats, seeds 0–9 → 100 fold-runs → 15,990 pooled predictions
- Class imbalance: class_weight='balanced' on SVM and RF — our own explicit choice; the paper states none
- Eq. (1) is ambiguous, so BOTH readings are implemented; the soft vote is the headline because it is the literal reading
- Every number in this deck is written to results/tables/*.csv by that one command — nothing typed by hand

No data leakage — a bug I found and fixed in my own code

An earlier version min-max scaled all 1,599 rows once, then cross-validated. The scaling constants were therefore computed using test-fold rows. Fixing it — moving MinMaxScaler inside the Pipeline — lowered my scores. The numbers on the next slides are from the corrected version.

Results — model ranking

Pooled over 100 fold-runs (10-fold CV × 10 repeats)



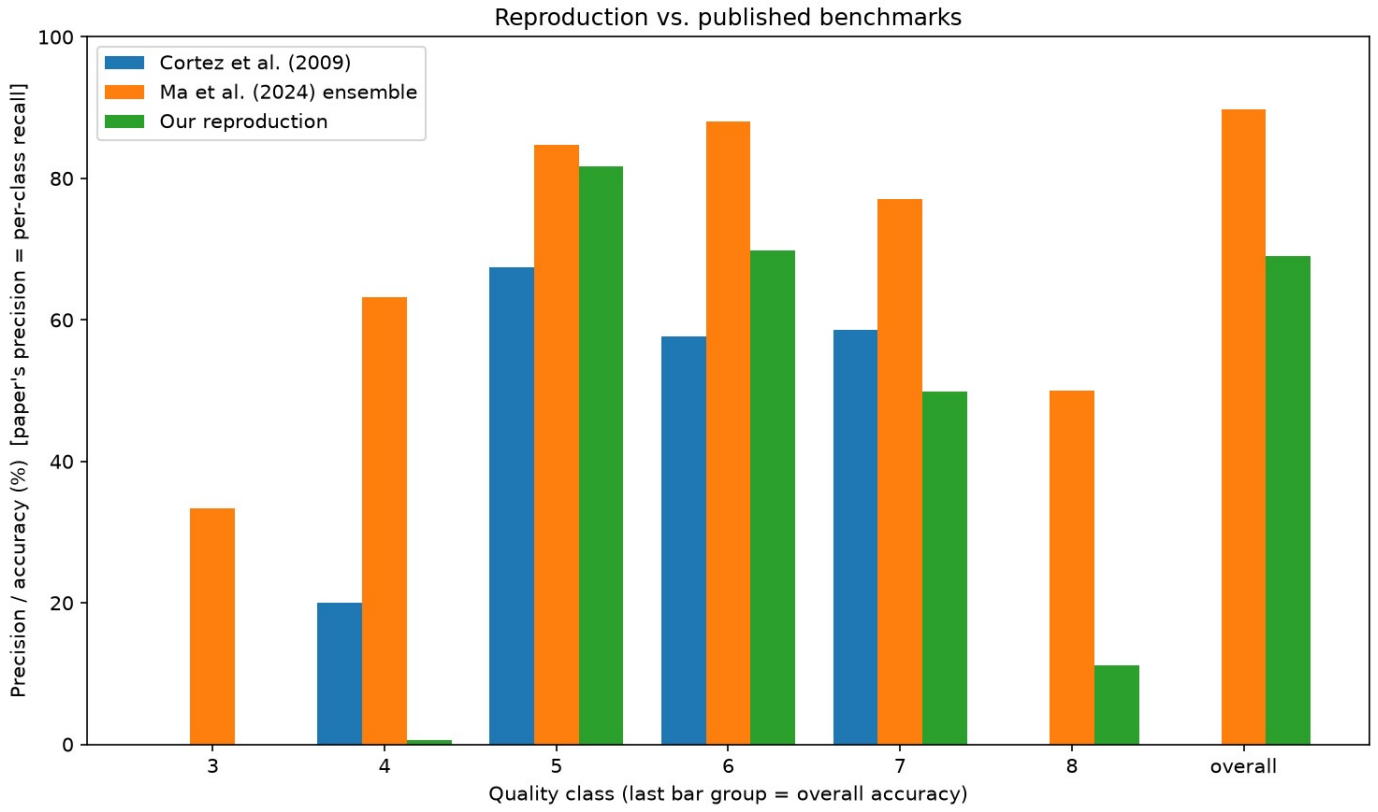
−2.02

POINTS: OUR ENSEMBLE vs RANDOM FOREST

- Our ensemble beats 3 of its 4 members — but trails Random Forest
- The paper reports the opposite: their ensemble is 9.02 points ABOVE their best member
- Measured weights span only 0.2356 – 0.2698 — a maximum deviation of 0.0198 from a uniform 0.25

Results — per class, against both benchmarks

Per-class values use the paper's own metric definition



Class	Cortez	Ma	Ours
3	0.00	33.33	0.00
4	20.00	63.16	0.57
5	67.50	84.80	81.66
6	57.70	88.06	69.87
7	58.60	77.08	49.90
8	0.00	50.00	11.11
Overall	—	**89.73**	**69.01**

The shape reproduces: mid-tier classes strong, extremes hardest — the paper's own conclusion. On class 5 we are within ~3 points.

Terminology finding: the paper calls this metric "precision", but its own Eq. (14) divides by the ACTUAL-class total — that is recall.

Why 89.73% does not reproduce

Three independent checks, all computed from the paper's own printed tables

1 Protocol mismatch

Their Table 9 confusion matrix sums to 491 samples — 30.7% of 1,599. That is a single hold-out split, not the 100-run cross-validation the text describes. Ours scores all 1,599 samples ten times over.

2 Internal arithmetic

The diagonal of that same Table 9 is 403 of 491 = 82.08%, which is 7.65 points below the 89.73% stated in the paper's own text.

3 The weights are flat

Their Table 8 weights are never more than 0.053 from a uniform 0.25. Applied to their OWN base accuracies they give 67.52%.

Weighted average of the paper's own base accuracies	**67.52%**
Our reproduction → 1.49 points above it	**69.01%**
Paper's reported ensemble → 22.21 points above it	**89.73%**

Mechanism: with weights this flat, a weighted sum ≈ an unweighted average — which lands BETWEEN its members, never above the strongest.

Caveat: the Table 9 cells were transcribed by eye from the printed PDF and may contain reading errors — two of the six per-class values do not reproduce, most plausibly for that reason. This is reported as "we could not reconcile these figures", NOT as a claim that the paper is incorrect.

Conclusion

Structurally the reproduction succeeded
All four base learners, Eqs. (1) and (4)–(9), min-max preprocessing and the 10×10 CV protocol were implemented as specified.

Numerically it did not
69.01% against a reported 89.73%, and the ensemble does not overtake Random Forest.

The gap is diagnosed, not excused.

Methodological lesson: an accuracy-and-diversity weighting scheme that normalises across learners of similar error rates produces weights too flat for a strong learner to dominate — so on an imbalanced multiclass problem it averages toward its members rather than exceeding the best of them.

Four contributions

1. A leak-free, single-command reproducible implementation
2. A three-check diagnosis of the discrepancy, from the paper's own tables
3. A terminology correction: the paper's "precision" is numerically recall
4. Both readings of an ambiguous Eq. (1) implemented and compared

Future work and references

Future work

- Imbalance: SMOTE / ADASYN or cost-sensitive loss, targeted at classes 3, 4 and 8
- Statistical rigour: paired t-test or Wilcoxon across the 100 fold-runs — currently point estimates, no confidence intervals
- Weight sharpening: add a temperature to Eqs. (5) and (9) to test the flat-weight diagnosis directly — it predicts the ensemble should overtake RF once weights are sharp enough
- Generalisation: repeat on the 4,898-sample white wine set (already in data/raw/)
- Obtain the publisher's machine-readable Table 9 to remove the transcription caveat

References

[1] Ma, R., Mao, D., Cao, D., Luo, S., Gupta, S., & Wang, Y. (2024). From vineyard to table: Uncovering wine quality for sales management through machine learning. Journal of Business Research, 176, 114576.

[2] Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. Decision Support Systems, 47(4), 547–553.

[3] UCI Machine Learning Repository — Wine Quality Dataset.

[4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.

Thank you — questions welcome.

Backup 1 — Confusion matrices

Pooled over 100 fold-runs (15,990 predictions)

Ensemble (soft vote)

true \ pred	3	4	5	6	7	8
3	0	9	70	21	0	0
4	1	3	372	144	10	0
5	4	11	5561	1202	31	1
6	3	17	1533	4458	360	9
7	0	0	114	874	993	9
8	0	0	0	96	64	20

Random Forest

true \ pred	3	4	5	6	7	8
3	0	2	80	18	0	0
4	8	0	344	169	9	0
5	0	11	5582	1201	16	0
6	0	12	1346	4740	282	0
7	0	0	89	883	1016	2
8	0	0	0	90	70	20

Row sums are 10× the dataset class counts because predictions are pooled across the 10 repeats (1,599 × 10 = 15,990). Class 4 at 0.57% is literally 3 correct out of 530; class 8 at 11.11% is 20 out of 180.

Backup 2 — Full per-class metrics

Conventional precision, recall and specificity (%)

Model	Metric	3	4	5	6	7	8
Ensemble (soft)	precision	0.00	7.50	72.69	65.61	68.11	51.28
	recall	0.00	0.57	81.66	69.87	49.90	11.11
	specificity	99.95	99.76	77.24	75.68	96.68	99.88
Random Forest	precision	0.00	0.00	75.02	66.75	72.94	90.91
	recall	0.00	0.00	81.97	74.29	51.06	11.11
GradBoost	recall	0.00	2.64	78.41	68.75	53.27	11.11
SVM	recall	0.00	0.00	76.81	65.19	25.93	0.00
MLP	recall	2.00	0.94	74.99	61.39	39.15	0.00

The paper reports only its "precision" metric, which is numerically the recall row above. Full tables: results/tables/metrics_*.csv

Backup 3 — Optional unsupervised extension

Not part of the reproduction; kept behind a --with-novelty flag

- Clustering (K-means / Ward, k = 3..8): best silhouette 0.211 at K-means k=3 — but k=6 and k=7 are within 0.001, so it is effectively a tie
- Outlier detection: Isolation Forest and LOF both flag class 3 at 20.0%, roughly twice the ~7–10% of classes 5 and 6 — the rare classes are genuinely atypical, not merely few
- Class 8 is method-dependent — 33.3% under Isolation Forest but 5.6% under LOF
- Cluster-ID fed back as a 12th feature: NO improvement on the three target classes (3, 4, 8 all exactly 0.00 change); classes 5 and 6 declined, class 7 rose 0.50 — reported as a negative result
- FP-Growth association rules recover all four attributes the paper highlights (alcohol, sulphates, volatile acidity, citric acid), plus five more

Note: the cluster-feedback comparison was produced by a 2-repeat smoke run, not the 10-repeat canonical protocol, so its baseline column differs slightly from the headline per-class table.

Backup 4 — Table 9 reconciliation detail

Recomputing the paper's per-class values from its own confusion matrix

Class	Paper reports	Recomputed column-wise	Recomputed row-wise	Column-wise match?
3	33.33	33.33	33.33	yes (exact)
4	63.16	63.16	54.55	yes (exact)
5	84.80	84.80	86.93	yes (exact)
6	88.06	88.50	87.62	within 0.6
7	77.08	61.67	60.66	no
8	50.00	60.00	75.00	no

- 4 of 6 reproduce column-wise (3 of them exactly); only 1 of 6 reproduces row-wise
- Column-wise = TP ÷ actual-class total = recall — confirming the terminology finding empirically
- The two non-matching classes (7 and 8) are most plausibly my own transcription errors from the printed PDF